More Statistical Sins & Introduction to Machine Learning

John Guttag

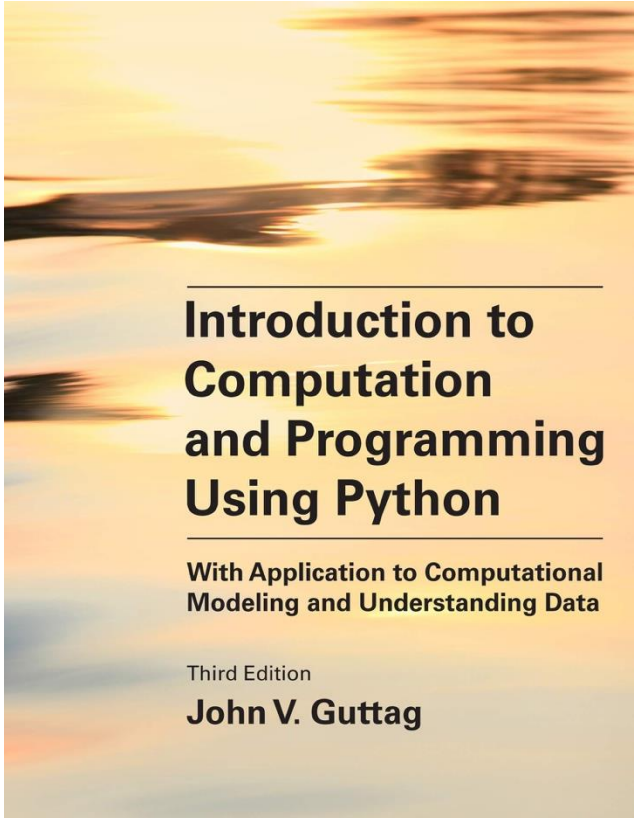
MIT Department of Electrical Engineering and Computer Science

Announcements

- Today is penultimate lecture
- Wednesday final lecture, and release of final problem set
- April 29, last quiz
- May 6, problem set 5 due

Assigned Reading

- Today and Wednesday:
  - Based on Chapters 20, 24, 26

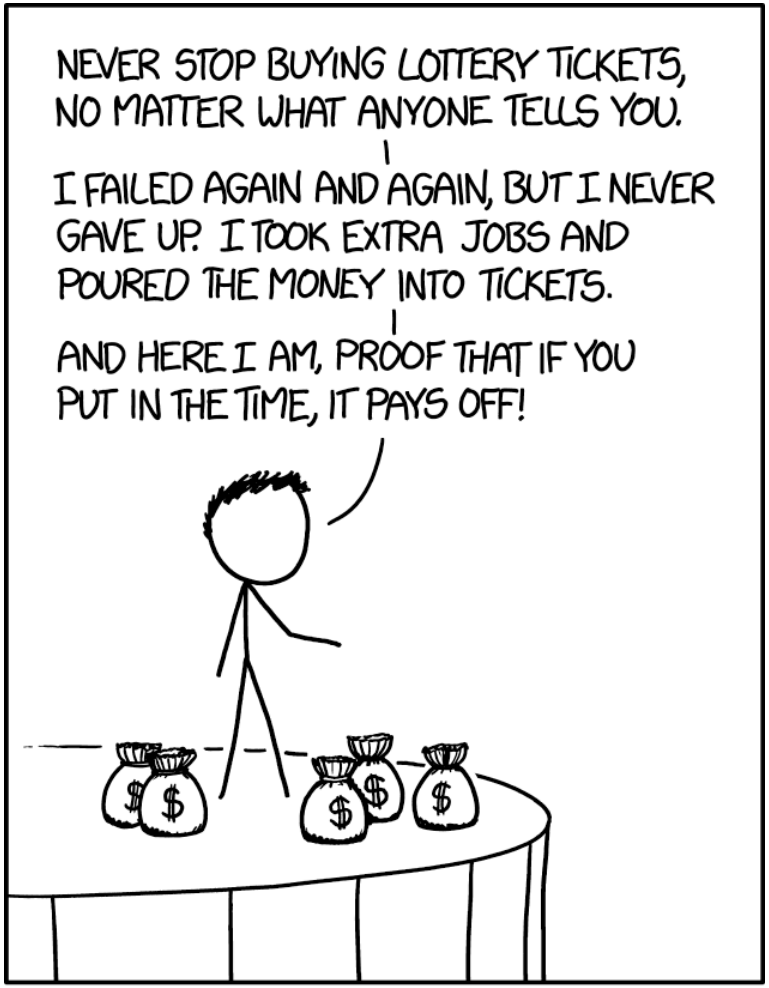


We've Talked a Lot About Sampling

- All statistical techniques are based upon the assumption that by sampling a subset of a population we can infer things about the population as a whole
- As we have seen, *if random sampling is used*, one can make meaningful mathematical statements about the expected relation of the sample to the entire population
- Easy to get random samples in simulations
- Not so easy in the field, where some examples are more convenient to acquire than others

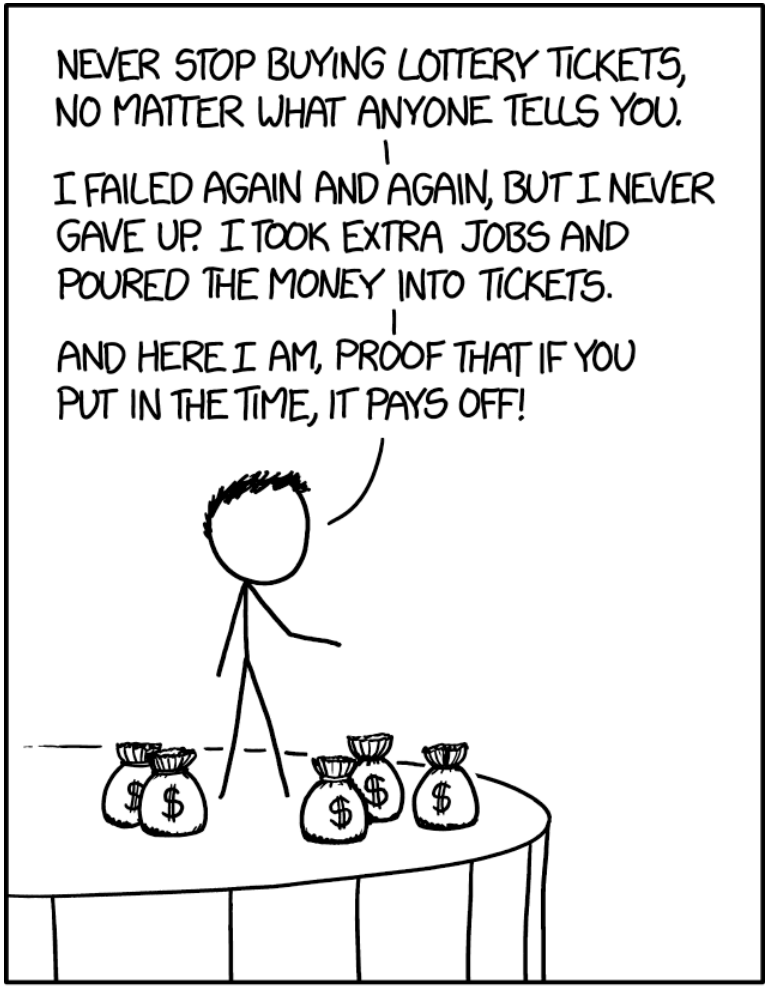
Non-representative Sampling

- "Convenience sampling" not usually random, e.g.,
  - Survivor bias, e.g., course evaluations at end of course or grading final quiz in 6.100B on a strict curve
  - Non-response bias, e.g., opinion polls conducted by mail or online



Non-representative Sampling

- "Convenience sampling" not usually random, e.g.,
  - Survivor bias, e.g., course evaluations at end of course or grading final exam in 6.100B on a strict curve
  - Non-response bias, e.g., opinion polls conducted by mail or online



EVERY INSPIRATIONAL SPEECH BY SOMEONE SUCCESSFUL SHOULD HAVE TO START WITH A DISCLAIMER ABOUT SURVIVORSHIP BIAS.

Non-representative Sampling

- When samples are not random and independent, we can still do things like compute means and standard deviations
- **But we should be very wary of using statistical techniques to draw conclusions from them**
- Moral: Understand how data was collected, and whether assumptions used in the analysis are satisfied

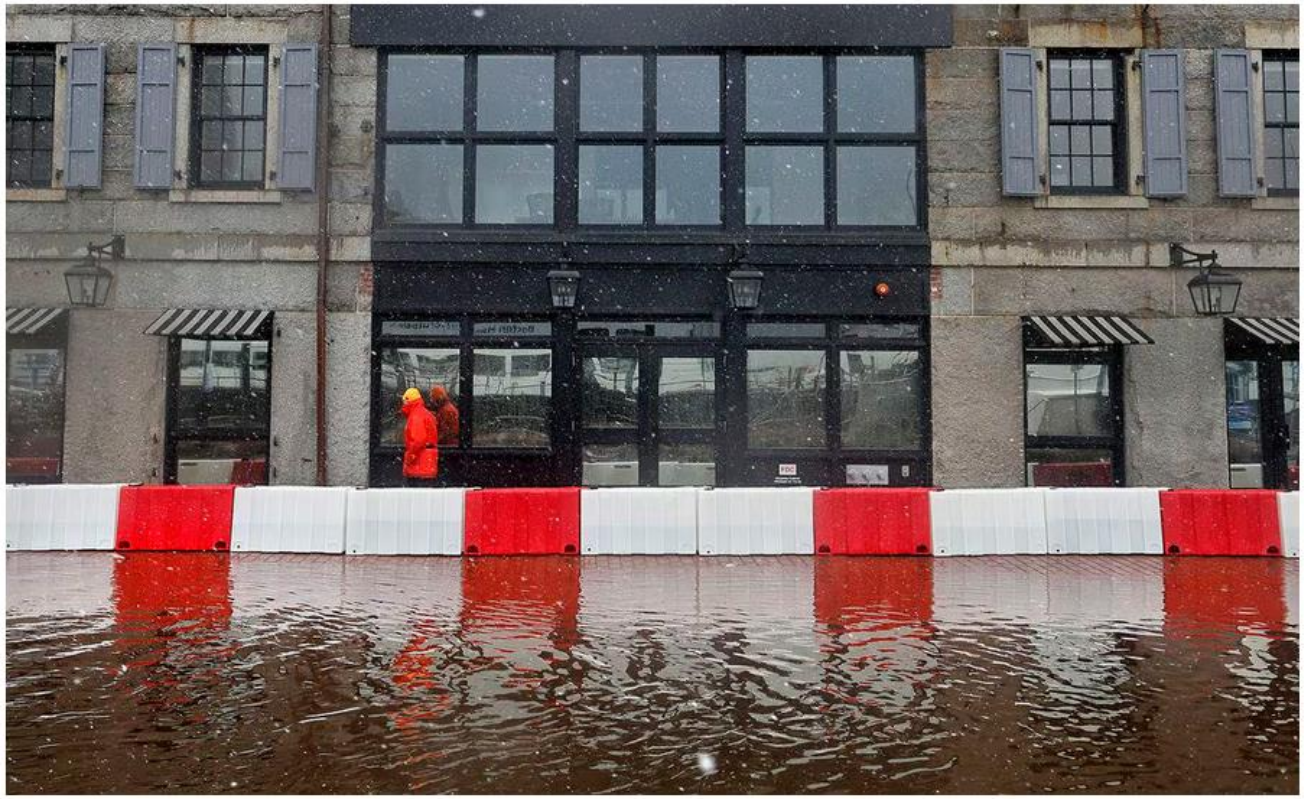
From the *Boston Globe*

As sea level rise accelerates, Boston is 'deep into overtime' to build coastal protections by 2030

Sea level rise in Boston broke a record in 2023. It's expected to get worse soon.

By Erin Douglas Globe Staff, Updated May 3, 2024, 5:20 p.m.

92



Flood barriers kept the water away from a building on Long Wharf during a winter storm along Boston's waterfront. Due to increasing sea levels, many areas of the city will need additional flood protections by 2030. LANE TURNER/GLOBE STAFF

The Counter Argument

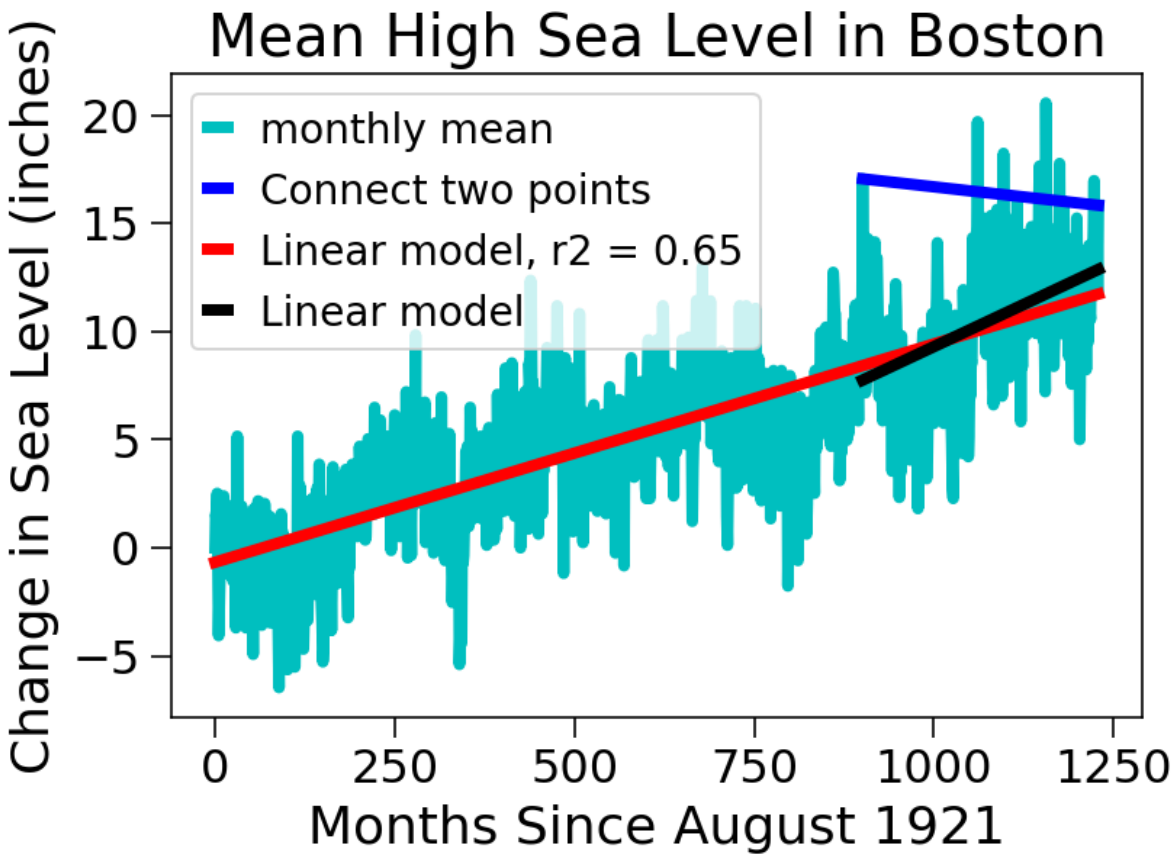
In the last 327 months preceding this article, the high-water level in Boston **dropped** almost 1.25 inches!

Is this true?

Fact: the average water level in April 2024 was 1.25 inches lower than in September 1996.

The Full Story

Moral: Look at intervals, not points



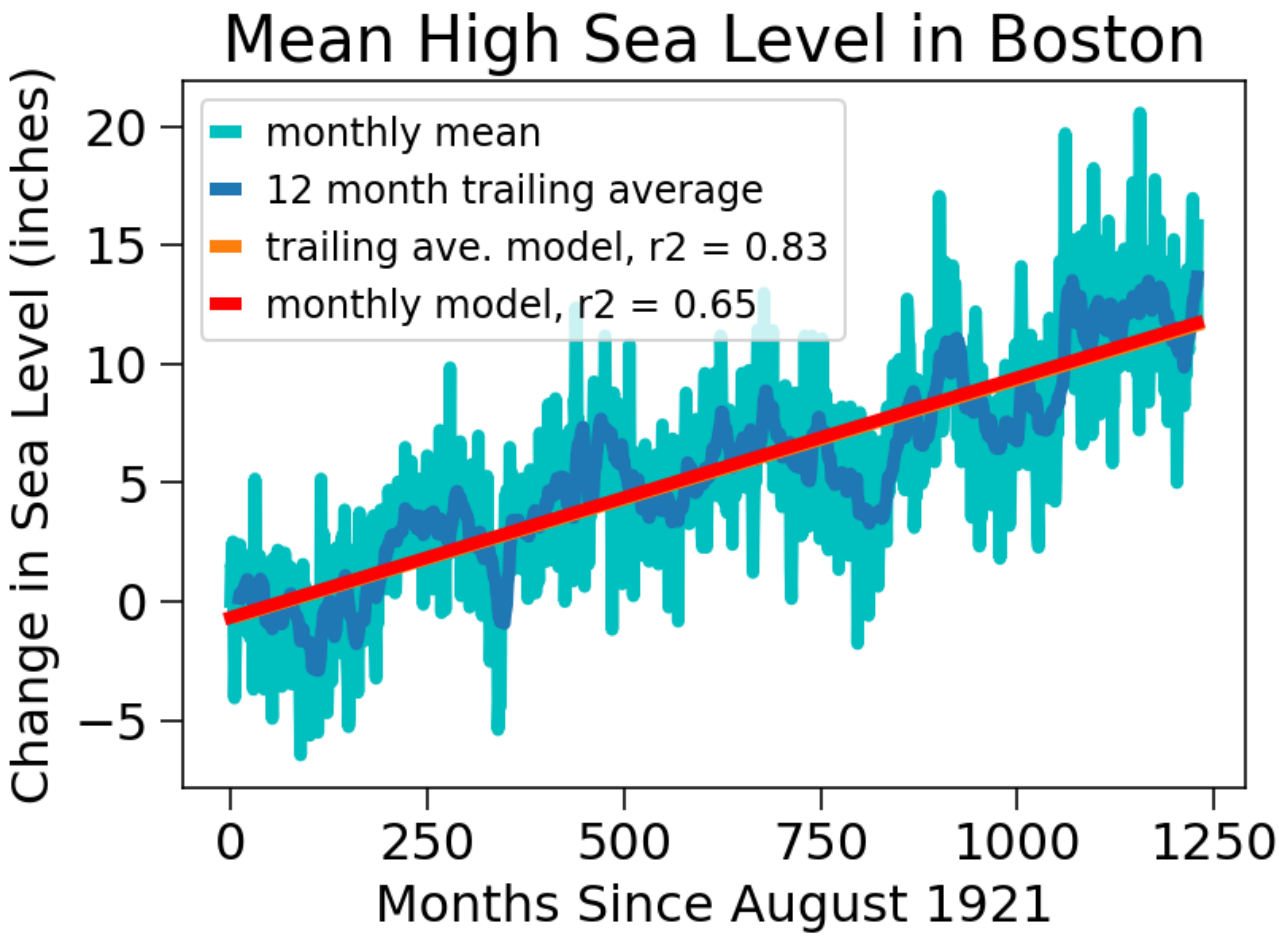
Moral: Don't confuse fluctuations with trends

Moral

Beware of

Slope of linear fit actually steeper since Sept. 1996

Sometimes Useful to Smooth Data



What do we see better when smoothed?

What can't we see when we smooth?

Linear model of trailing ave. = [0.01004759 -0.71193985]
Linear model of monthly data = [0.01008333 -0.6650442]

Successful Treatment of Kidney Stones

Success Rate

	Treatment A	Treatment B
Small stones	93%	87%
Large stones	73%	60%
All	**78%**	**83%**

C. R. Charig; D. R. Webb; S. R. Payne; J. E. Wickham

It appears that A is better for **both** large and small stones,
but worse in aggregate

Huh?

Successful Treatment of Kidney Stones

	Treatment A	Treatment B
Small stones	93% (81/87)	87% (234/270)
Large stones	73% (192/263)	60% (55/80)
Combined	78% (273/350)	83% (289/350)

C. R. Charig; D. R. Webb; S. R. Payne; J. E. Wickham

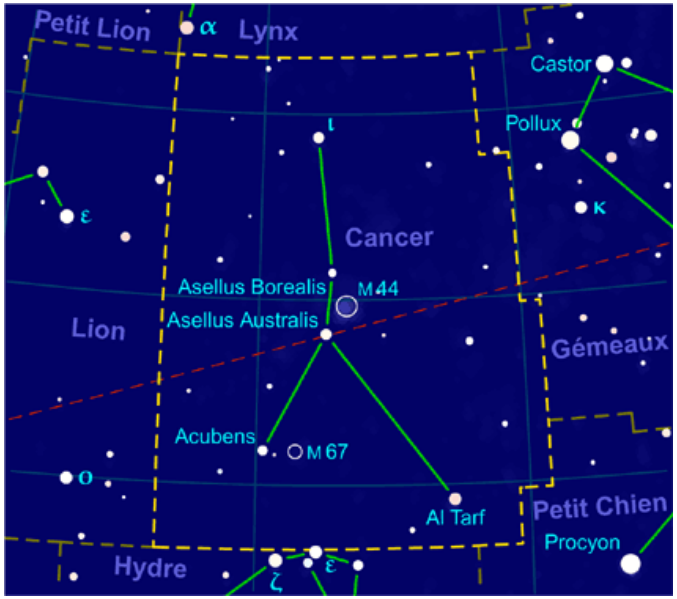
Treatment A more often applied to hard cases

Simpson's paradox: trend appears in each sub-group, but disappears when groups are aggregated.

Moral: Be careful when aggregating groups

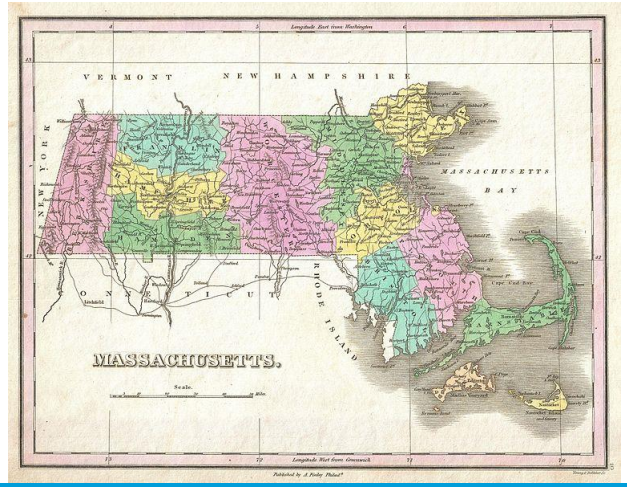
Cancer Clusters

- A **cancer cluster** is defined by the CDC as “a greater-than-expected number of cancer cases that occurs within a group of people in a geographic area over a period of time”
- About 1000 “cancer clusters” per year are reported to health authorities in the U.S.
- Vast majority, but not all, are deemed not significant



A Hypothetical Example

- Massachusetts is about 10,000 square miles
- About 36,000 new cancer cases per year
- An attorney partitioned state into 1000 regions of 10 squares miles each, and looked at distribution of cases
- In the most recent 5-year period, region 111 had
  - 39, 39, 37, 44, 42 new cases
- How worried should residents be?
  - How likely just bad luck



https://commons.wikimedia.org/wiki/File:1827_Finley_Map_of_Massachusetts_-_Geographicus_-_Massachusetts-finley-1827.jpg

How Do We Answer that Question?

- One-sample t-test
  - How does mean of sample relate to some hypothesized mean
- Test statistic: $\dfrac{\mu s - \mu h}{\sigma\sqrt{\dfrac{1}{n}}}$

  μh = hypothesized mean

  Standard error
- Mean number of cases/region:
  - μh = 36,000/1,000 = 36

```
t_stat, p_value = stats.ttest_1samp([39, 39, 37, 44, 42],
                                    popmean = 36)
print(f'p-value = {p_value:.3f}')
```

p-value = 0.028 Looks highly statistically significant

Or Does It?

- We looked at this in the last lecture
- If the attorney had hypothesized that Region 111 was a cancer cluster, and then looked at the data
  - The conclusion about significance would have been sound
- But the attorney looked at 1,000 regions and then cherry picked the one with the most cases
- This is an example of what?
- What should we do about it?
- To reach significance level of 0.05, we need a p-value of 0.05/1,000, and 0.028 is a very long way from that!

The Bottom Line

- When drawing inferences from data, skepticism is merited.
- But remember, skepticism and denial of facts is different.
- "Doubt, indulged and cherished, is in danger of becoming denial; but if honest, and bent on thorough investigation, it may soon lead to full establishment of the truth." – Ambrose Bierce

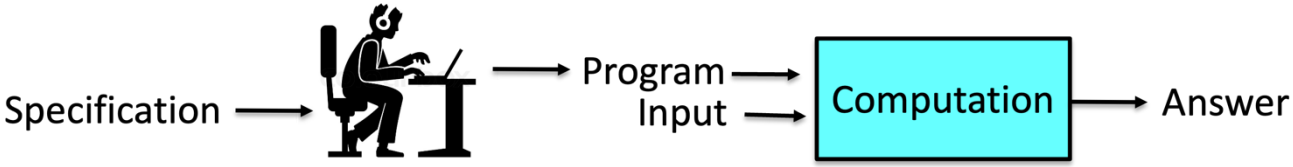
Moving On to Machine Learning

AI Is Everywhere

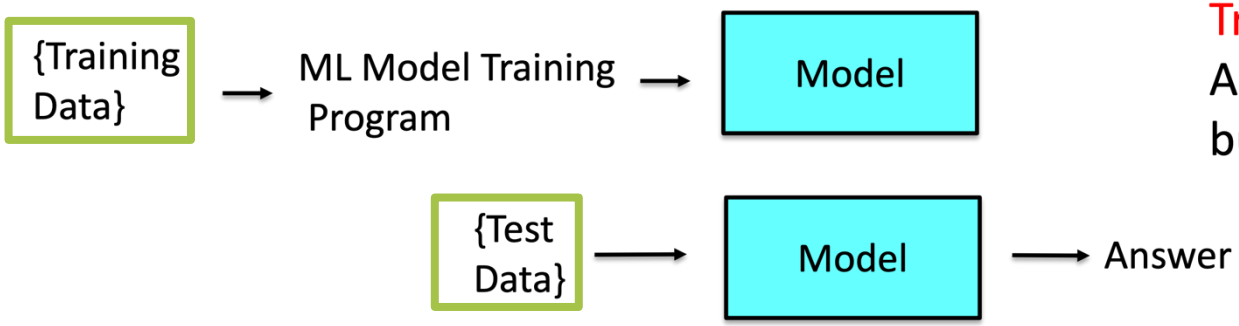
- It drives cars
- It writes news stories
- It write programs
- It reads medical images
- It recognizes speech
- It generates photos and videos
- It designs drugs
- Etc.

And modern AI is all about machine learning (ML)

What Is Machine Learning



Machine Learning



Training time
Analogous to building a program

Inference time
Analogous to using a program

Based on statistics, but quite different from traditional statistics

Traditional Statistics

- Form a hypothesis, e.g., vaccines reduce severity of Covid
- Gather factual and counterfactual data
- See if there is a difference in the two populations
  - While attempting to control for confounders
- Some challenges
  - Inventing hypotheses
  - Controlling for confounders in high dimensional data
  - Extrapolation to other datasets
  - Data typically numerical

Machine Learning

- Gather lots of data
  - Of many types, e.g., numerical, text, video
- Search for interesting hypotheses in the data
- Challenges
  - Often very computationally intensive
  - Models often “black boxes”
  - Over-fitting can be hard to detect
    - Out-of-domain data

It's All Based Upon

The Unreasonable Effectiveness of Data

Alon Halevy, Peter Norvig, and Fernando Pereira, Google

Birds-eye View of Machine Learning

Classification	Clustering, Association
fit a model, e.g., (image→ bird species), where prediction is from a discrete set of classes)	divide examples into groups based on their similarity E.g., split customers by their purchasing habits to do targeted marketing
Regression	**Other**
fit a model, e.g., (Weight → Spring Displacement), and use that function to predict displacement for other weights	generative AI

Supervised learning: given (Input, Answer) pairs, build a model that predicts Answer from Input. Answers are called **labels**

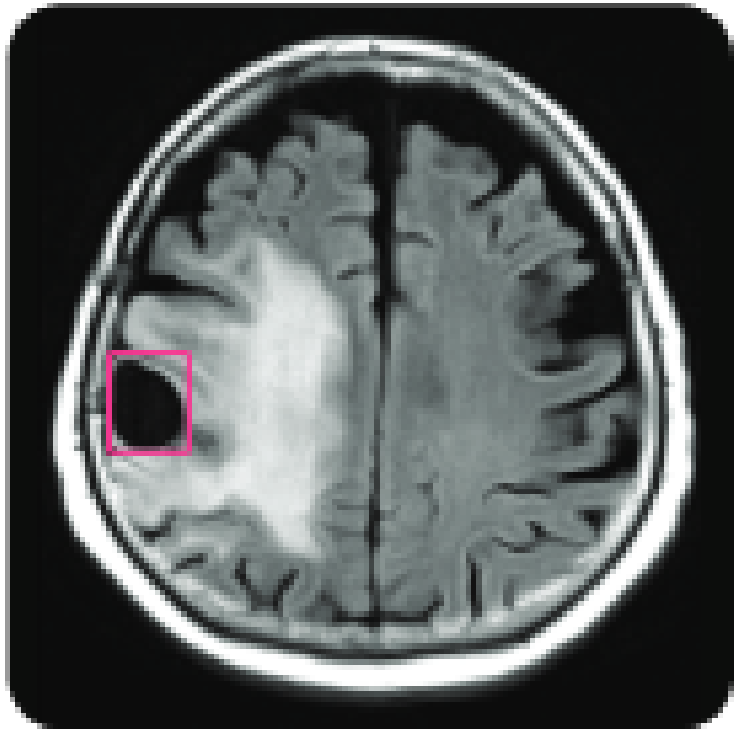
Unsupervised learning: build a descriptive model of data
No answers are given

Classification: Candidate for Knee Replacement?

Yes

No

Regression: How Big Is the Tumor

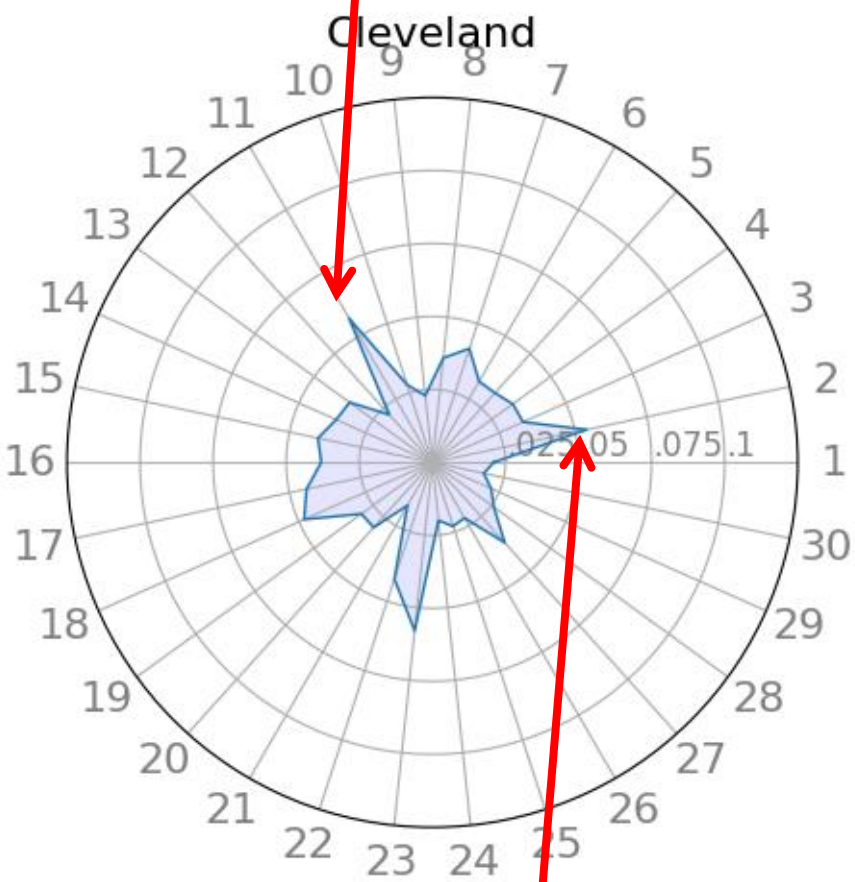


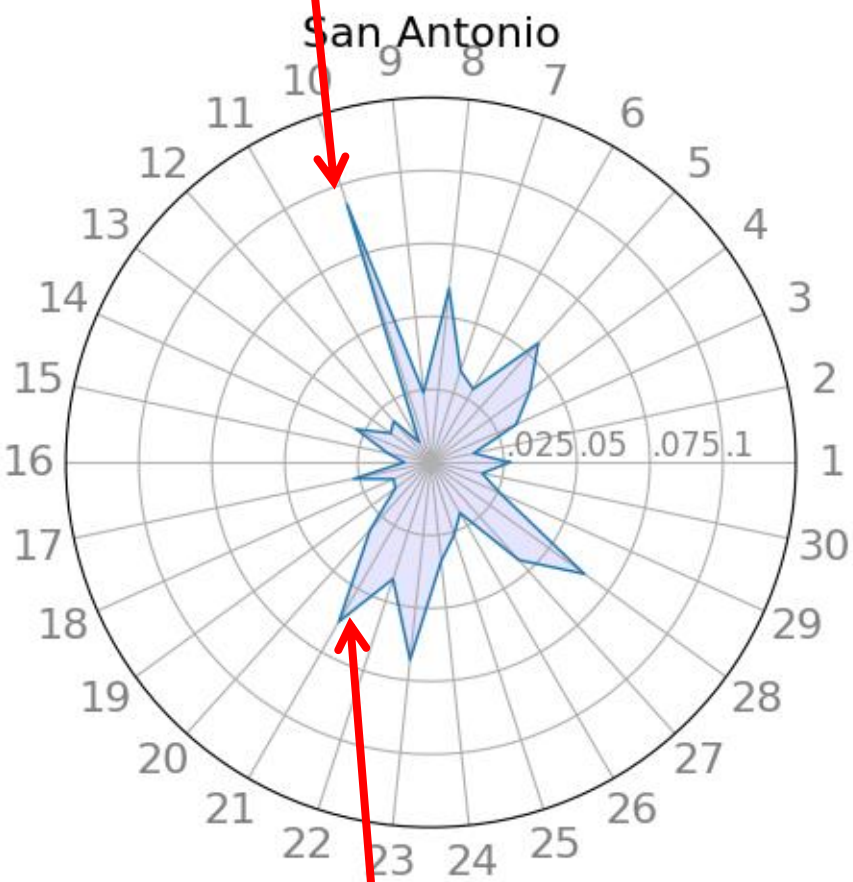
22.7 x 25.6 x 28.5 mm

Clustering: Clustering Possessions

Pick and roll on the right side

Pass to left elbow, drive to the basket or pass to perimeter





Pick and roll at the top of the key

Passing and cycling around the perimeter

Generative Models

Learn structure of examples in training data

At inference time generate **new** examples from same distribution

Aitana López is an AI-generated creation by a Spanish agency that grew tired of booking real models.

Prompt: Write a 50-word review of 6.100B

Response: 6.100B provides a solid foundation in computational thinking and data science. Topics include plotting, stochastic programs, probability, and optimization. A valuable course for those interested in coding and real-world problem-solving.

Stochastics underlies it all
Source of “creativity”

Chatbots

- Based on a model called a Generative Pre-Trained Transformer (supervised learning)
- Basic idea is given a sequence of tokens, e.g., words, predict the next word in the sequence
  - E.g.,: `A bird in ???`
  - Once we've predicted the next word, we can reinvoke the model to continue to predict output:
  - E.g.,: `A bird in the ???`
- Non-deterministic

Two Reviews of 6.100B

Prompt: Write a 50-word review of 6.100B

6.100B is a challenging, engaging course that builds strong intuition through simulation and modeling. John Guttag delivers clear, thoughtful lectures. Problem sets are demanding but rewarding. Students gain practical skills and a deeper understanding of data, randomness, and computational approaches to real-world problems.

6.100B is a demanding but rewarding course that emphasizes simulation, modeling, and clear thinking. Lectures by John Guttag are engaging and practical. Problem sets are time-intensive but deepen understanding. Students leave with strong intuition for data science and computational problem solving.

Did It Actually Watch the Lectures?

Prompt: Does John Guttag give out candy in 6.100B

There's **no strong, widely documented evidence** (in official materials or reliable sources) that John Guttag routinely hands out candy in 6.100B.

What *can* be verified is indirect:

- In at least one lecture, he explicitly mentions forgetting a "candy bag," which suggests it **has happened at least occasionally**.
- However, course materials, syllabi, and official descriptions make **no mention of candy or rewards**—they focus on simulations, modeling, and problem sets.

introcomp.mit.edu +1

So the most accurate answer is:

- **Yes, possibly (and likely occasionally)**
- **No, not a formal or consistently documented practice**

In other words, it seems like a small, informal classroom quirk—not a defining or guaranteed feature of 6.100B.

Chatbots Are a Triumph of Scale

- Models use large neural nets with many layers and a large input history
- One based on a model called GPT-5
  - Input history of upto 128k tokens
  - 5-10 trillion parameters (compare to polynomial regression)
  - 6 months to train on 10's of thousands of high-end GPUs
  - Cost maybe $500M
- Models trained on massive amounts of data
  - ~1M times more than a human sees in their lifetime

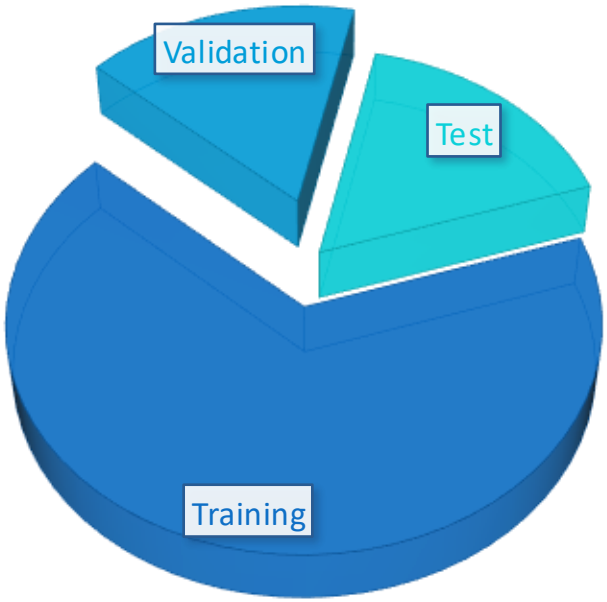
Actual numbers not publicly available.
These are my guesses.

Birds-eye View of Machine Learning

Classification fit a model, e.g., (image→ bird species), where prediction is from a discrete set of classes)	Clustering, Association divide examples into groups based on their similarity E.g., split customers by their purchasing habits to do targeted marketing
Regression fit a model, e.g., (Weight → Spring Displacement), and use that function to predict displacement for other weights	Other (e.g., LLMs, dimensionality reduction, GANs,…)

Evaluating Classifiers: Training/Testing Methodology (revisited)

- Split dataset into groups, ideally:
  - Training set: used to train the model
  - Validation set: used during training to select good meta parameter values for a specific model
  - Test set: used to estimate the performance of the model

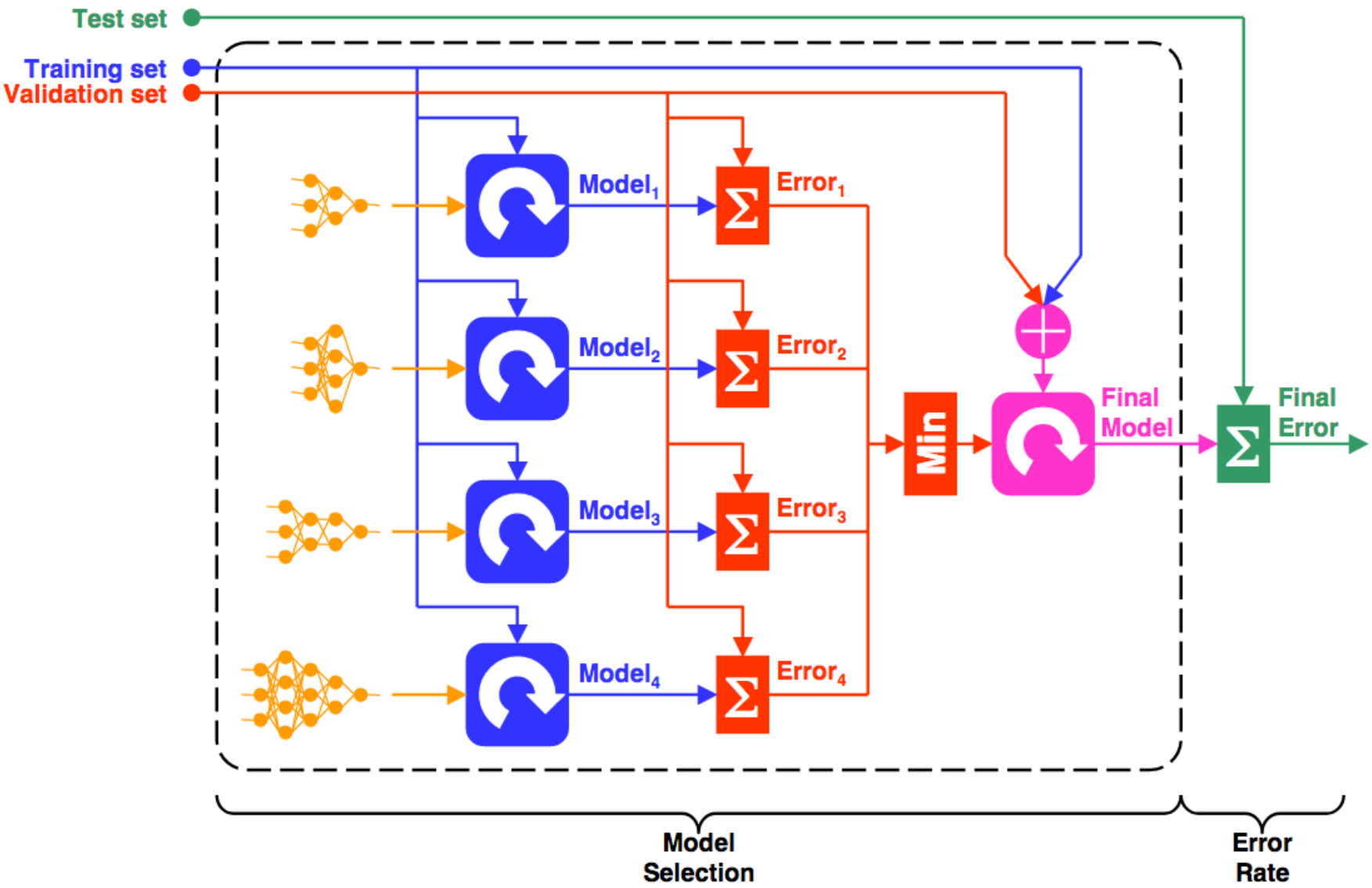


Why a Validation Set (or Sets)

- Almost all modeling techniques have free parameters:
  - Degree for linear regression
  - Number of trees and depth for random forests
  - Etc.
- Split training data into multiple training and validation sets
- Search for “optimal” parameter values

Warning: can lead to misleading estimates of error rates if there is not also a holdout set for testing

The Process


Test set
Training set
Validation set
Model$_1$
Model$_2$
Model$_3$
Model$_4$
Error$_1$
Error$_2$
Error$_3$
Error$_4$
Min
Final Model
Final Error
Model Selection
Error Rate

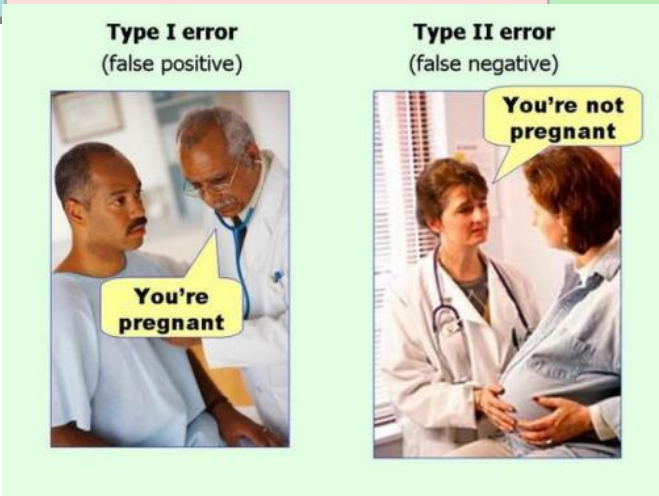
Evaluating Classifiers

- It's trickier that it might seem
- Why is it not sufficient to just look at the fraction of correct predictions, i.e., accuracy
- Consider a disease that occurs in 1% of population
  - Predicting disease-free has an accuracy of 0.99

A Fuller Picture of Performance

Confusion Matrix

		predicted condition	
	total population	prediction positive	prediction negative
true condition	condition positive	**True Positive (TP)**	**False Negative (FN)** (type II error)
	condition negative	**False Positive (FP)** (Type I error)	**True Negative (TN)**



Sensitivity and Specificity

$$sensitivity = \frac{true\ positive}{true\ positive + false\ negative}$$

True positive rate

$$specificity = \frac{true\ negative}{true\ negative + false\ positive}$$

True negative rate

Sensitivity: don't miss positive examples
Specificity: don't label negative examples as positive

sensitivity = recall

Predictive Value

- So far, analyzed wrt to true label
- Now, wrt prediction

$$Positive\ Predictive\ Value = \frac{true\ positives}{true\ positives + false\ positives}$$

How worried should I be by a positive test?

$$Negative\ Predictive\ Value = \frac{true\ negatives}{true\ negatives + false\ negatives}$$

How reassured should I be by a negative test?

For Example

$$Positive\ Predictive\ Value = \frac{true\ positives}{true\ positives + false\ positives}$$

$$Negative\ Predictive\ Value = \frac{true\ negatives}{true\ negatives + false\ negatives}$$

Tested positive: 14 had strep, 1 did not
Tested negative: 14 had strep, 71 did not

$$PPV = 14/15 \approx .933$$

$$NPV = 71/85 \approx .84$$

Rapid Strep Test
Wikipedia

Common Metrics (Recap)

$$sensitivity = \frac{true\ positive}{true\ positive + false\ negative}$$

How many positive cases did I identify?

$$specificity = \frac{true\ negative}{true\ negative + false\ positive}$$

How many negative cases did I identify?

What percentage of claimed positives are right?

$$positive\ predictive\ value = \frac{true\ positive}{true\ positive + false\ positive}$$

What percentage of claimed negatives are right?

$$negative\ predictive\ value = \frac{true\ negative}{true\ negative + false\ negative}$$

We can often trade these off, by adjusting some threshold to favor more false negatives or false positives

Matters especially when classes are unbalanced

Break Time

Which Metric is Most Important?

- It depends on the setting

For AI in a driverless car trying to detect pedestrians, we probably want very few false negatives

For AI trying to detect cheating on an exam, we probably want very few false positives

When designing ML algorithms, we can typically trade one for the other

Many Kinds of Classifiers to Choose From

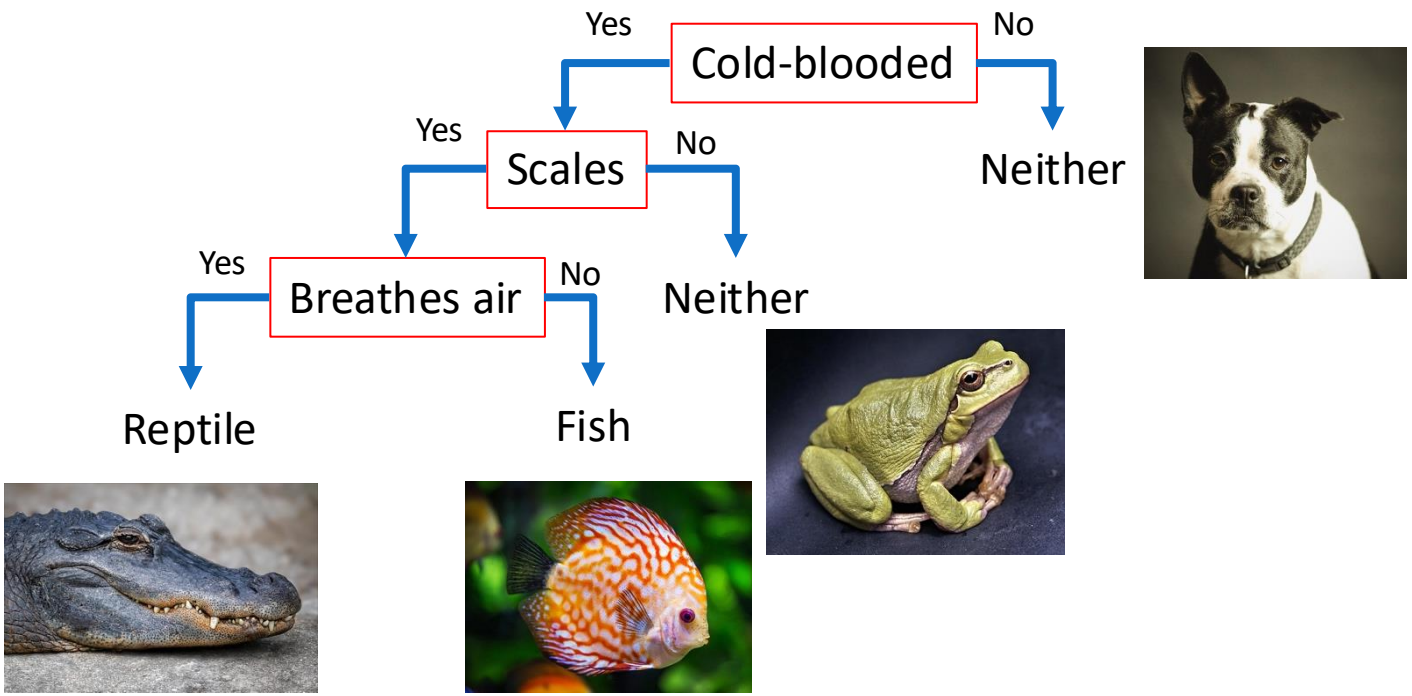
- K-nearest neighbor
- Support Vector Machines
- Decision Trees
- Random Forrest
- Boosted Decision Trees
- Logistic Regression
- Naïve Bayes
- Bayesian network
- RBMs
- Deep Learning based techniques (CNNs, RNNs, Transformers, …)
- ….

Slide credit: D. Hoiem

Many Different Kinds of Classifier Models

- The best choice typically depends upon the kind of data
- Neural networks usually the right choice for
  - Images/signals (e.g., computer vision)
  - Sequences (e.g., language)
- Random forests often the right choice for tabular data

Decision Tree: Fish, Reptile, or Neither



Will He Go Golfing?

- Based on the weather forecast, predict whether someone is likely to play
- Data format:

```
Format: [[features, label]]

Features: [outlook, temperature, humidity, windy]

        outlook ∈ {sunny, overcast, rainy}

        temperature ∈ {hot, mild, cool]}

        humidity ∈ {high, normal}

        windy ∈ {true, false}
```

David's Recent History of Decisions

```
david_history = [
    [['Sunny', 'Hot', 'High', 'False'], 'No'],
    [['Sunny', 'Hot', 'High', 'True'], 'No'],
    [['Overcast', 'Hot', 'High', 'False'], 'Yes'],
    [['Rainy', 'Mild', 'High', 'False'], 'Yes'],
    [['Rainy', 'Cool', 'Normal', 'False'], 'Yes'],
    [['Rainy', 'Cool', 'Normal', 'True'], 'No'],
    [['Overcast', 'Cool', 'Normal', 'True'], 'Yes'],
    [['Sunny', 'Mild', 'High', 'False'], 'No'],
    [['Sunny', 'Cool', 'Normal', 'False'], 'Yes'],
    [['Rainy', 'Mild', 'Normal', 'False'], 'Yes'],
    [['Sunny', 'Mild', 'Normal', 'True'], 'Yes'],
    [['Overcast', 'Mild', 'High', 'True'], 'Yes'],
    [['Overcast', 'Hot', 'Normal', 'False'], 'Yes'],
    [['Rainy', 'Mild', 'High', 'True'], 'No'],
]
```

John's Recent History of Decisions

```
john_history = [
    [['Sunny', 'Hot', 'High', 'False'], 'No'],
    [['Sunny', 'Hot', 'High', 'True'], 'No'],
    [['Overcast', 'Hot', 'High', 'False'], 'Yes'],
    [['Rainy', 'Mild', 'High', 'False'], 'Yes'],
    [['Rainy', 'Cool', 'Normal', 'False'], 'No'],
    [['Rainy', 'Cool', 'Normal', 'True'], 'No'],
    [['Overcast', 'Cool', 'Normal', 'True'], 'No'],
    [['Sunny', 'Mild', 'High', 'False'], 'No'],
    [['Sunny', 'Cool', 'Normal', 'False'], 'No'],
    [['Rainy', 'Mild', 'Normal', 'False'], 'Yes'],
    [['Sunny', 'Mild', 'Normal', 'True'], 'Yes'],
    [['Overcast', 'Mild', 'High', 'True'], 'Yes'],
    [['Overcast', 'Hot', 'Normal', 'False'], 'Yes'],
    [['Rainy', 'Mild', 'High', 'True'], 'No'],
    ]
```

No rather than Yes

The Two Trees

<u>The decision tree learned for David</u>

```
if outlook == Sunny:
    if humidity == High:
        Predict: No
    if humidity == Normal:
        Predict: Yes
if outlook == Overcast:
    Predict: Yes
if outlook == Rainy:
    if windy == False:
        Predict: Yes
    if windy == True:
        Predict: No
```

Notice that two trees are quite different

<u>The decision tree learned for John</u>

```
if temperature == Hot:
    if outlook == Sunny:
        Predict: No
    if outlook == Overcast:
        Predict: Yes
if temperature == Mild:
    if humidity == High:
        if outlook == Rainy:
            if windy == False:
                Predict: Yes
            if windy == True:
                Predict: No
        if outlook == Sunny:
            Predict: No
        if outlook == Overcast:
            Predict: Yes
    if humidity == Normal:
        Predict: Yes
if temperature == Cool:
    Predict: No
```

Applying Each Tree to Same Example

- ['Rainy', 'Cool', 'High', 'False’] – This example is not in training data
- Prediction for sample using David tree: Yes
- Prediction for sample using John tree: No

- Models were trained using same code, but on different data
- Led to models that reached different conclusions about the same example
- Moral: the training data matters a lot

Why a Different First Node?

<u>David</u>

if outlook == Sunny

…

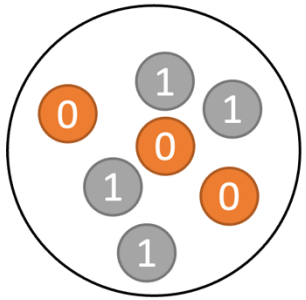
<u>John</u>

if temperature == Hot:

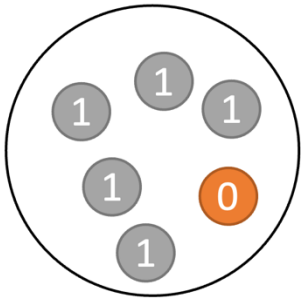
…

- Learning algorithm looks to find the feature that supplies the most information about the label
- I.e., when data is partitioned into subsets using that feature, how pure is each subset with respect to the label

Less Pure



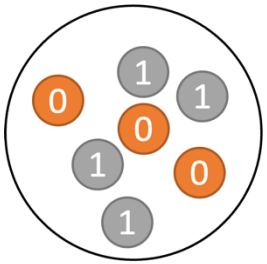
More Pure



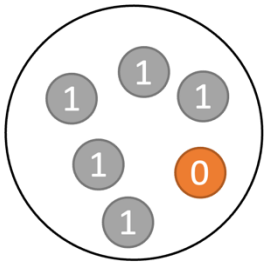
Learning a Decision Tree

- Which attribute do we choose at each level?
- The one with the highest information gain
  - The one that reduces the uncertainty/impurity the most
- We want to determine which attribute in a given set of training feature vectors is most useful for discriminating between the classes to be learned.
- Information gain tells us how important a given attribute of the feature vectors is.
- We will use it to decide the choice and ordering of attributes in the nodes of a decision tree.

Less Pure



More Pure



Building a Decision Tree (ID3 Algorithm)

- For now, assume attributes are categorical
- Choose the attribute with the highest information gain
- Create branches for each value of attribute
- Examples partitioned based on selected attributes
- Repeat with remaining attributes
- Stopping conditions
  - All examples assigned the same label
  - No examples left

It's a greedy algorithm

Not guaranteed to find optimal tree

What About Numeric Attributes?

- Can turn into categorical attributes by binning
  - As we do when creating a histogram
  - Once binned, we treat each bin as its own attribute
- Consider every possible binary partition (linear in number of distinct numeric values)
  - Choose the partition with the highest gain
  - Works fine when there are are reasonable number of partitions, e.g., age (in years) in our example
  - Computationally infeasible if there are are too many ways to partition
    - Look at subset of possible partitions (e.g., age in years rather than days)

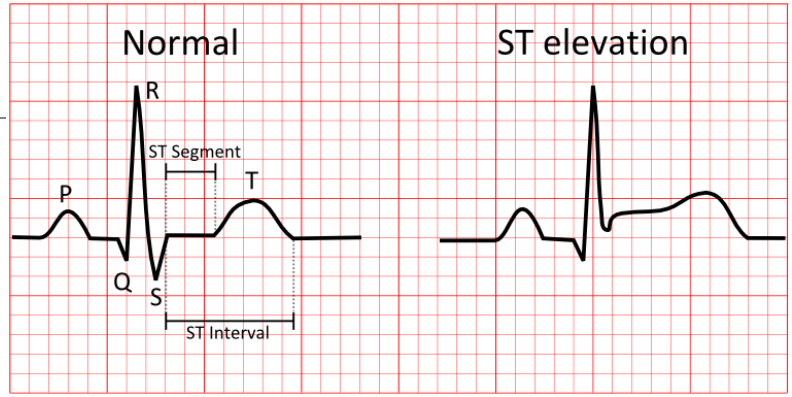
Native Python Code for Building Tree in lect12.py

- Functions
  - entropy
  - info_gain
  - best_split
  - majority_class
  - build_tree
  - classify
  - pretty_print_tree
- About 70 lines of code in total
- In practice, most people use version in sklearn
  - As I will from here on
- But I wanted to give you a chance to see what is involved

How Many Different Possible Feature Vectors for Golf Task

- # outlooks * # temperatures * # humidities * # windys = 36
- If we had 36 distinct examples for each of David and John, could we just memorize answers?
- Yes, but
  - In most situations, number of possible examples much higher than number of training examples
  - Mapping from features to labels often not 1-1
- Goal of ML training: Build a model that
  - Makes good predictions on data not seen during training
  - Makes a good guess (based on probability) when label is ambiguous

A More Interesting Example



ECG (Heart) Wave Form

- Data
  - 250 patients
    - Heart rate in beats per minute
    - Number of past heart attacks
    - Age
    - ST elevation (binary)
- Outcome (death) based on features
  - Probabilistic, not deterministic
  - E.g., older people with multiple heart attacks at higher risk
- Goal: **Classify** based on outcome - so we can predict if new patients are going to survive

Data Sample

patient	heartRate	stElev	age	numAttacks	outcome
0	84	0	55	0	0
1	76	0	80	0	0
2	50	0	80	0	0
3	60	0	44	0	0
4	106	0	77	1	0
5	40	0	33	0	0
6	63	0	46	0	0
7	78	0	61	1	1
8	68	0	63	1	0
9	69	0	82	1	0
10	87	0	73	0	0
11	62	0	58	0	0
12	73	1	70	2	0
13	55	0	81	0	0
14	85	0	60	1	1
15	52	0	55	1	1
16	56	0	53	0	0
17	62	0	74	0	0
18	53	0	59	0	0
19	71	0	41	1	1
20	97	0	51	0	1

1 means mortality (death)

Features | **Label**

Getting Into Features

- When building a model, often useful to inspect data to develop some intuition about it
- One common approach is to start by looking at correlation of individual features with the outcome

```
df = pd.read_csv('cardiacData.txt')
for feature in df.columns[:-1]:
    cor = df[feature].corr(df['outcome'])
    print(f'Correlation of {feature} with outcome = {cor:.2f}')
```

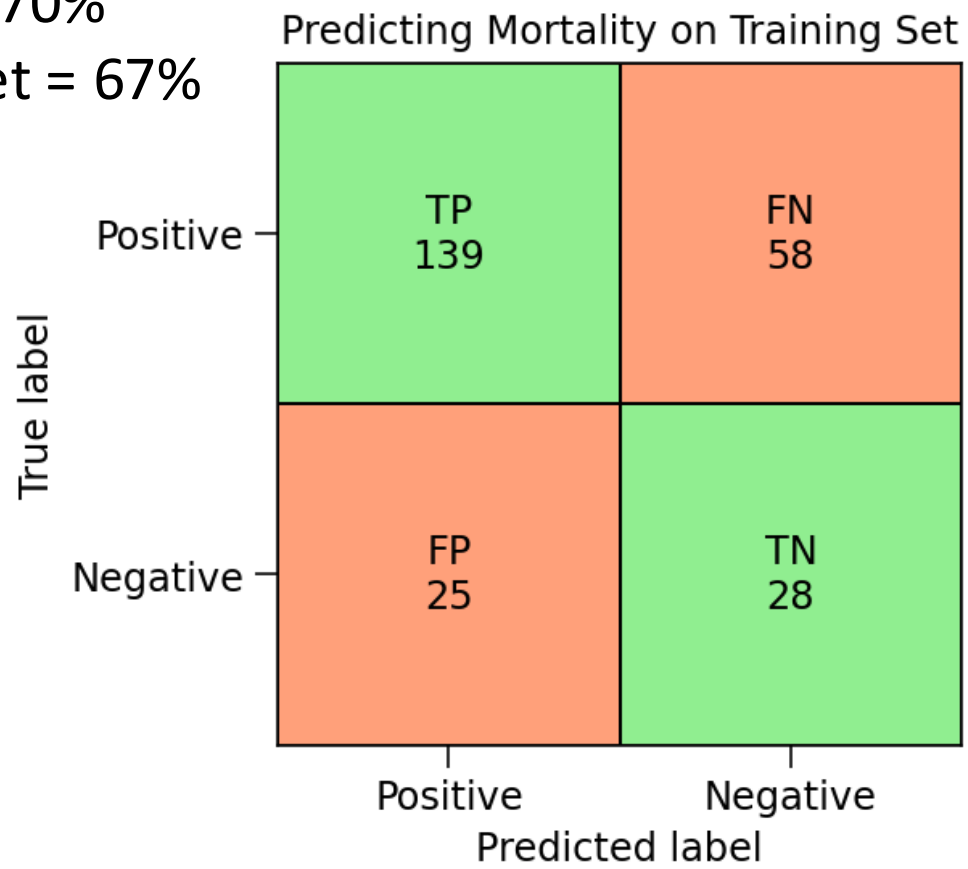
Correlation of heartRate with outcome = 0.02
Correlation of stElev with outcome = 0.26
Correlation of age with outcome = 0.05
Correlation of numAttacks with outcome = 0.48

Try a Very Simple Decision Tree (one rule)

IF numAttacks >= 1 THEN died = yes ELSE died = no

Mean accuracy on training set = 79%
Mean recall on training set = 70%
Mean precision on training set = 67%

Let's use sklearn to try deeper trees

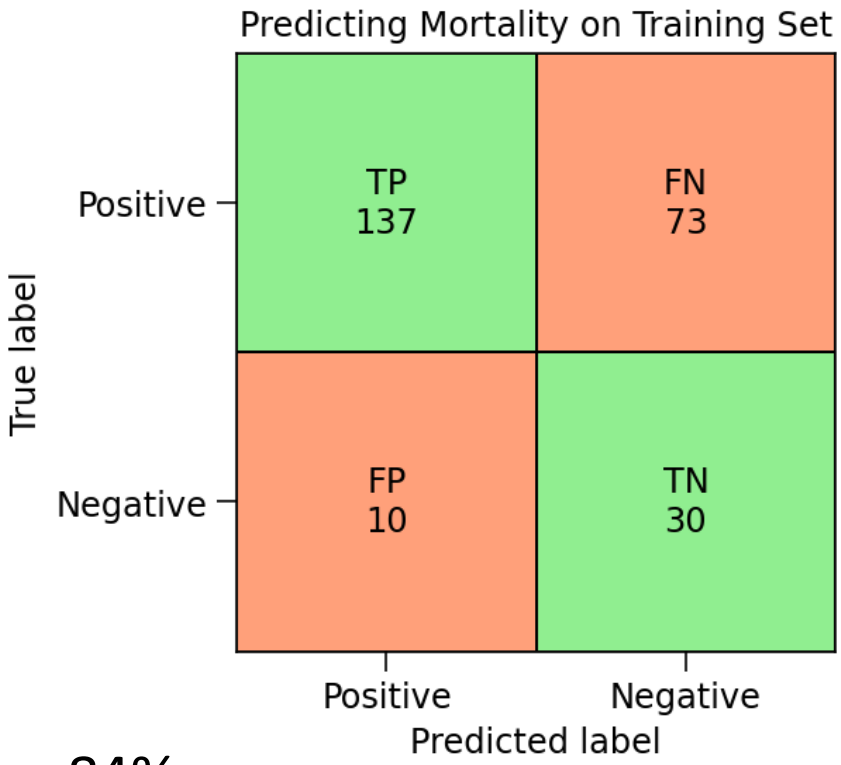


sklearn

- sklearn is a python library that implements many machine learning functions
  - Including random forests and many more
  - Plus helper functions to do many ML tasks
- In most cases, you want to use sklearn, not your own implementation
  - Implementations are fast, robust, and generally choose the right defaults for various parameters and hyperparameters
- Final problem set will give you experience using it
- Used in today's .py file

A Tree of Depth 2 Learned Using sklearn

```
|--- numAttacks <= 0.50
|   |--- stElev <= 0.50
|   |   |--- class: 0.0
|   |--- stElev >  0.50
|   |   |--- class: 1.0
|--- numAttacks >  0.50
|   |--- heartRate <= 54.00
|   |   |--- class: 1.0
|   |--- heartRate >  54.00
|   |   |--- class: 1.0
```



Mean accuracy on training set = 84%

Mean recall on training set = 88%

Mean precision on training set = 71%

If 2 is better than 1, why stop there?

Models' Performance on Training Data

Tree of Depth 2

Mean accuracy on training set = 84%
Mean recall on training set = 88%
Mean precision on training set = 71%

Tree of Depth 12

Mean accuracy on training set = 95%
Mean recall on training set = 89%
Mean precision on training set = 95%

- Algorithm finds good decision tree for training data
- Risk is that it is highly optimized for that data, and won't work well on other data
  - **Overfitting**

Split Data Into Training and Test Sets

Tree of Depth 12	Tree of Depth 12
Mean accuracy on training set = 90%	Mean accuracy on test set = 80%
Mean recall on training set = 88%	Mean recall on test set = 68%
Mean precision on training set = 84%	Mean precision on test set = 68%

- Algorithm finds good decision tree for training data
- But it can overfit to training data, so that performance on training data is misleading
- One solution: random forests
  - Train multiple trees, each on randomly chosen subset of training data and random subset of features
  - At inference time, make a prediction using each tree
  - Combine predictions, e.g., by taking a vote